

```
$ spring run Functionallab --level=0..100
```

# Spring Boot × FP

// 從 0% (缺點全開) 到 100% (完美) 的體質改造

0%  
地雷區

25%  
Stream

50%  
純函數

75%  
Result

100%  
Core/Shell

\$ cat refactor-plan.md

# 今日路線：一段結帳邏輯的五次進化

## 0% / 病灶盤點

NPE、共享狀態、隱藏副作用

## 25% / 宣告式起步

Stream 與 Optional

## 50% / 純函數

record 不可變、測試零 mock

## 75% → 100% / 架構收尾

Result、組合、Core/Shell

目標：每往右一格，就少一類會半夜叫你起床的 bug。

PART 01 — 0%

# 命令式地雷區

先誠實面對多數專案的起點

# 每一行都是地雷

```
1  @Service
2  public class OrderService {
3      private double total; // 共享可變狀態
4
5      public Order checkout(Long id) {
6          Order order = repo.findById(id).get();
7          total = 0;
8          for (OrderItem item : order.getItems()) {
9              if (item != null
10                  && item.getPrice() != null) {
11                  total += item.getPrice()
12                      * item.getQty();
13              }
14          }
15          if (order.getCoupon() != null) {
16              // ...巢狀 if 打折...
17          }
18          order.setTotal(total);
19          emailService.send(order);
20          return repo.save(order);
21      }
22  }
```

## SMELL REPORT

- × `total` 欄位：單例 Service，兩個請求共用
- × 裸奔 `.get()`：NPE 看運氣
- × `null` 檢查防不完
- × 測「算錢」卻真的寄信

改任何一行都怕動全身

# 0% 的缺點清單

## NPE 隨時引爆

findById().get()、getPrice() 都可能是 null

## 併發直接壞掉

共享欄位 total，Service 預設單例

## 無法單獨測試

驗折扣要先 mock repository 和 email

## 改 A 壞 B

計算、狀態、I/O 全綁在一起

FP 的採用是一條光譜，不是開關 —— 一格一格往右走

PART 02 — 25%

# Stream 與 Optional

不動架構，只用 JDK 內建工具

# 迴圈 → 管線

```
1 double total = order.getItems().stream()
2   .filter(Objects::nonNull)
3   .mapToDouble(i -> i.getPrice() * i.getQty())
4   .sum();
```

## 命令式

手寫「怎麼做」：迴圈、累加、防呆

## 宣告式

說「做什麼」：過濾、映射、加總

# null 判斷 → Optional 管線

```
1 Order order = orderRepository.findById(id)
2   .orElseThrow(() -> new OrderNotFoundException(id));
3
4 double discounted = Optional.ofNullable(order.getCoupon())
5   .filter(Coupon::isValid)
6   .map(c -> total * 0.9)
7   .orElse(total);
```

## 消滅了

裸奔 .get()、巢狀 null 檢查、共享欄位

## 還沒解決

order 仍可變、計算仍和 I/O 綁在一起

map / filter 就是 FP 的 Functor：資料進容器，用函數描述變換。

PART 03 — 50%

# 不可變資料與純函數

本次改造投資報酬率最高的一步

# 把「計算」抽離「流程」

```
1 public record Pricing(BigDecimal subtotal, BigDecimal total) {
2
3     public final class PricingRules { // 純函數
4         static Pricing price(
5             List<OrderItem> items,
6             Optional<Coupon> coupon) {
7
8             BigDecimal subtotal = /* stream 加總 */;
9             BigDecimal total = coupon
10                 .filter(Coupon::isValid)
11                 .map(c -> subtotal.multiply(RATE_90))
12                 .orElse(subtotal);
13             return new Pricing(subtotal, total);
14         }
15     }
```

## Service 變薄

```
1 public Order checkout(Long id) {
2     Order order = findOrder(id);
3     Pricing pricing = PricingRules.price(
4         order.items(), order.coupon());
5     return repo.save(
6         order.withTotal(pricing.total()));
7 }
```

取資料 → 純計算 → 存回去

record 沒有 setter ; 純函數不碰 DB、不寄信、不看時鐘

# 測試的解放

## 0% 的測試

```
1 @SpringBootTest
2 class OrderServiceTest {
3     @MockBean OrderRepository repo;
4     @MockBean EmailService email;
5     // ...啟動容器、stub 回傳值...
6     // 只是想驗「打九折」
7 }
```

## 50% 的測試

```
1 @Test
2 void 有效優惠券打九折() {
3     var items = List.of(
4         new OrderItem("鍵盤",
5             new BigDecimal("1000"), 1));
6
7     Pricing p = PricingRules.price(
8         items, Optional.of(validCoupon()));
9
10    assertThat(p.total())
11        .isEqualByComparingTo("900");
12 }
```

零 mock · 零容器 · 毫秒級

# 動手拉拉看

```
$ spring run Functionallab
```

● 危險

## 0% 傳統命令式



- ✗ 到處 null 檢查，NPE 隨時引爆
- ✗ setter 滿天飛，狀態隨時被偷改
- ✗ 計算混著 I/O，測試要 mock 一堆
- ✗ 用例外控制流程，呼叫端全靠猜
- ✗ 副作用散落各層，改一行怕動全身

PART 04 — 75%

# Result 型別與函數組合

錯誤與規則都變成「值」

# 錯誤寫進型別

```
1 public sealed interface Result<T> {  
2     record Ok<T>(T value) implements Result<T> {}  
3     record Err<T>(String reason) implements Result<T> {}  
4 }  
5  
6 // 呼叫端 switch 窮舉：漏接錯誤直接編譯不過  
7 return switch (result) {  
8     case Ok<Pricing>(Pricing p) -> ResponseEntity.ok(p);  
9     case Err<Pricing>(String reason) ->  
10         ResponseEntity.badRequest().body(reason);  
11 };
```

比「希望大家記得 try-catch」可靠得多 — 錯誤處理從紀律問題變成編譯問題。

# 商業規則 = 可拼裝的管線

```
1  static UnaryOperator<Pricing> memberDiscount(Member m) {  
2      return p -> m.isVip()  
3          ? p.withTotal(p.total().multiply(RATE_95))  
4          : p;  
5  }  
6  
7  Function<Pricing, Pricing> rules =  
8      memberDiscount(member)  
9          .andThen(couponDiscount(coupon))  
10         .andThen(freeShippingOver(new BigDecimal("1000")));
```

## 加規則

多寫一個函數、多 andThen 一次

## 不用改舊程式

開放封閉原則的 FP 版

PART 05 — 100%

# Functional Core, Imperative Shell

核心絕對純粹，副作用住在邊界

# 殼與核心

```
1  @Service
2  public class CheckoutService { // Shell
3      public CheckoutResponse checkout(Long id) {
4          Order order = repo.findById(id) // I/O
5              .orElseThrow(...);
6
7          CheckoutDecision decision = CheckoutCore
8              .decide(order.snapshot(), clock.instant());
9
10         return switch (decision) {
11             case Approved a -> {
12                 repo.save(a.pricedOrder());
13                 events.publish(new OrderPriced(a));
14                 yield CheckoutResponse.ok(a);
15             }
16             case Rejected r ->
17                 CheckoutResponse.fail(r.reason());
18         };
19     }
20 }
```

## 核心全是純函數

```
1  public final class CheckoutCore {
2      public static CheckoutDecision decide(
3          OrderSnapshot order, Instant now) {
4
5          if (order.items().isEmpty())
6              return new Rejected("空訂單");
7
8          Pricing pricing = /* 純計算 */;
9          return new Approved(
10              order.withPricing(pricing));
11     }
12 }
```

### 兩個關鍵設計

1. 時間也是輸入：核心不偷看時鐘
2. 核心回傳「決定」，殼負責「執行」

# 0% vs 100%

| 指標      | 0%            | 100%                   |
|---------|---------------|------------------------|
| 可測試性    | mock 三個依賴才能開測 | 核心零 mock，毫秒級           |
| Null 安全 | NPE 看運氣       | Optional + record 寫進型別 |
| 可預測性    | 取決於當下狀態       | 相同輸入永遠相同輸出             |
| 併發安全    | 共享可變欄位，必壞     | 不可變資料，天生安全             |
| 錯誤處理    | 例外亂飛，全靠猜      | Result + switch 窮舉     |
| 擴充規則    | 往巢狀 if 裡塞     | 多一個函數、多一次 andThen      |

# 常見疑慮

## Spring 到處是副作用，衝突嗎？

互補：Spring 管殼（DI、交易、事件），FP 管核心。@Transactional 包在 Shell 上，Core 不知道交易存在。

## 一直 new 物件，效能爆嗎？

JVM 對短命小物件極快（TLAB + 分代 GC）。先換來正確性，壓測有熱點再談。

## 一定要走到 100%？

每一格都是穩定停靠點：25% 已消滅 NPE 大宗；50% 讓核心測試零 mock。

# 今日重點

**25%**

Stream 宣告式、Optional 進型別

**50%**

record 不可變、純函數零 mock

**75%**

Result 編譯期把關、andThen 組規則

**100%**

副作用住邊界，核心絕對純粹

完美不是「沒有副作用」，是副作用全部住在邊界。

# Refactor complete.

下一步：互動版文章 `/springboot/fp-in-spring-boot`

```
$ spring run Functionallab --level=100
```

```
stream → record → result → core/shell
```